

SDP



The Agentic Journey

Incremental Path from Issue to PR Automation

Start in 2-3 hours. Compound value at every phase. No SDLC rewire required.

"How do you move from experimenting with AI to trusting it in your SDLC?"

Most teams are stuck between all-in paralysis and experiment-forever drift.

The answer is a state machine: one label transition at a time.

Developer

Eliminate hours of issue triage, planning, and implementation boilerplate

30 min → 3 min

Triage time per issue with AI duplicate detection and auto-labeling

Tech Lead

Get structured, codebase-aware plans before any code is written

60% better estimates

When agents mine historical PRs before generating execution plans

Engineering Manager

See exactly where human control gates are and why they matter

2-3 hour setup

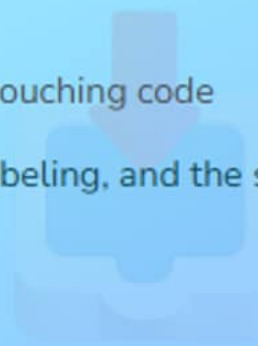
Copy 4 YAML files, configure secrets, create an issue — you are live

PART 1

Agentic Intake

Triage automation without touching code

Duplicate detection, auto-labeling, and the state machine spine

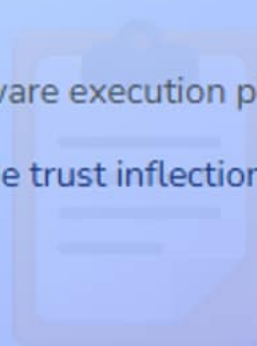


PART 2

Agentic Planning

Human-gated, codebase-aware execution plans

The /approve-plan gate is the trust inflection point

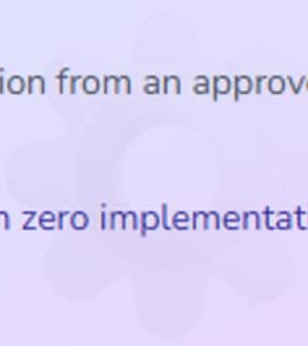


PART 3

Agentic Coding

Autonomous implementation from an approved plan

PR with tests appears with zero implementation keystrokes

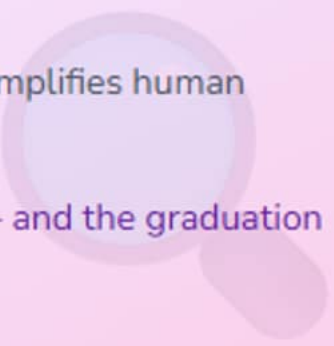


PART 4

Code Review

Automated analysis that amplifies human judgment

Security, logic, coverage — and the graduation exit ramp



Click any section to jump directly there

PART 1

Agentic Intake

Before touching a line of code, earn credibility by eliminating triage noise in 3 minutes.



State Machine Spine

new-issue → intake-complete: the full diagram lives here



Duplicate Detection

88% match found in 3 min, auto-labeled, fix version posted



20-30 min → 3 min

Triage time per issue, compounded across 14 dupes/month

Time to set up agentic intake from scratch

↳ 2-3 hours – copy 4 YAML files, configure secrets, open an issue

One Label Machine, Four Automation Phases

Every section of this talk advances exactly one state transition



Each phase's savings depend on the previous phase having run first — this ordering is not arbitrary

Issue Triage: From Investigation to Instant Classification

Manual Triage

20-30 min per issue

Developer reads, researches, decides priority

40 min per duplicate — full re-investigation

Inconsistent classification across team members

No link to historical context or fix versions

Agentic Intake

3 min per issue

AI classifies, routes, and checks duplicates

88% duplicate match detected and auto-labeled

Consistent classification driven by prompt template

Fix version posted automatically on duplicates

90%

faster triage

14×

duplicate ROI per month

3 hrs

total setup time

Duplicate Detection: 88% Match in 3 Minutes

14 duplicates/month × 40 minutes each — evaporating

```
$ github-actions[bot]
> "Triage new issue #4821: Login fails on mobile Safari"
🤖 Copilot (Intake Agent):
Analyzing issue type, priority, and codebase context ...
Searching for similar resolved issues in last 90 days ...

Found: Issue #4203 – 88% similarity score
Status: Resolved in v2.1.3 (2 weeks ago)
Fix: Corrected SameSite cookie attribute for Safari compatibility

✅ Label applied: duplicate
✅ Comment posted: Fix available in v2.1.3 – close if confirmed resolved
```

40 min saved · 14 duplicates/month · 560 min/month recovered

Agentic Planning

The `/approve-plan` gate is the single boundary between intake and autonomous coding.



Historical Research

Agent mines similar past PRs for codebase-aware planning



Execution Plan

Structured, reviewable plan generated before any code



`/approve-plan` Gate

Human types one comment; only then does code get written

Time to produce a codebase-aware execution plan
↳ 2-4 hrs → 15 min

Ad-Hoc Planning vs. Codebase-Aware Execution Plans

From intuition-based estimates to structured, historically-informed plans

✗ Traditional Planning

- 1 Read the issue
Understand requirements manually
- 2 Browse codebase
2-3 hours reading code and docs
- 3 Estimate from intuition
Hope you haven't missed edge cases
- 4 Start coding immediately
Discover wrong assumptions mid-sprint



40% of implementations hit mid-sprint scope changes

✓ Agentic Planning

- 1 Read the issue
Understand requirements manually
- 2 Agent mines past PRs
6 months of similar issues researched in minutes
- 3 Structured plan generated
Files, approach, tests — all specified up front
- 4 Human types /approve-plan
One comment — then autonomous coding begins



60% better estimate accuracy — approved plan is the contract

2-4 hrs of planning → 15 min with full historical context

Historical Context Changes the Math

Agents research what humans skip

60%

better estimate accuracy when agents mine past PRs before planning

6 months of similar issues

Agent queries resolved PRs matching the current issue type and scope

40% fewer mid-sprint surprises

Codebase-aware plans surface edge cases before coding starts

File-level specificity

Plan names exact files and functions to modify — no guessing during implementation

Test strategy included

Agent specifies test files and coverage targets in the plan before code is written



The plan is the artifact — not the code. Managers approve what gets built before anyone builds it.

From Context to Approved Plan: Three Distinct Steps



Historical Research

Agent mines similar past issues and resolved PRs for codebase-aware context

6 months of PR history

File-level change patterns

Past estimation accuracy



Execution Plan

Structured, reviewable plan posted as an issue comment before any code is written

Files to change — named

Test strategy specified

Risk areas flagged



/approve-plan Gate

Human reviews the plan and types /approve-plan — only then does any code get written

Explicit human opt-in

Full visibility before code

Rejection resets planning

The /approve-plan Gate: One Comment, Complete Control

✗ The Concern

Autonomous agents writing code sounds risky

What stops it from doing the wrong thing?

Managers need a clear control boundary

How do I reject a bad plan without breaking the workflow?

💡 The Gate

Agent posts execution plan as a readable issue comment

Human reads, questions, and optionally edits the plan

/approve-plan triggers coding — anything else does not

✓ The Outcome

State machine advances to plan-approved on human command only

Code is written exactly once — from an approved, reviewed plan

One comment = complete, auditable, reversible human control

100%

human-gated before code

Agentic Coding

An approved plan unlocks autonomous implementation — tests, PR, and description included.



Agent Assignment

Copilot coding agent picked up directly from the approved plan



Test Generation

Tests are a first-class output, not an afterthought



PR Creation

implementation-complete: PR opens, state machine advances

Time to implement from approved plan to open PR
↳ 8-12 hrs → 1-2 hrs

Three First-Class Outputs From One Approved Plan



Agent Assignment

Copilot coding agent is assigned directly from the approved plan — no manual kickoff required

Triggered by plan-approved label

Plan document is the instruction set

Runs in CI — no local env needed



Tests Included

Test generation is a first-class output — the plan specifies test strategy before coding begins

Unit and integration tests

Coverage targets from the plan

CI fails if tests are missing



PR Created

implementation-complete label fires as the PR opens — the state machine advances without human intervention

PR description auto-generated

Linked to source issue

Ready for automated review

Update Agent Behavior by Editing Markdown, Not Debugging YAML

⚠️ Embedded YAML

Instructions buried in workflow files

Hard to find, harder to review

Non-technical stakeholders locked out of prompt review

Prompt changes require YAML debugging

Risk of breaking CI triggers

No clear version history for agent behavior changes

✅ .github/prompts/

One markdown file per phase

triage-, planning-, implementation-, review-instructions.md

Product managers can read and improve prompts directly

Edit markdown — workflow YAML unchanged

Zero CI risk from prompt iteration

Prompts serve as living documentation of agent behavior

From 12-Hour Implementations to 2-Hour PRs

Manual Implementation

8-12 hours per task

Research, code, debug, write tests, write PR description

Tests written last — or skipped under deadline pressure

PR description written from memory of what changed

Context switching cost

Interruptions break implementation flow

Agentic Coding

1-2 hours per task

Coding agent implements from the approved plan

Tests generated as first-class output from the plan

PR description auto-generated with full context

Developer reviews, not builds

Human time spent on judgment, not keystrokes

85%

less implementation time

100%

of PRs include tests

0

implementation keystrokes



The approved plan is the implementation contract — the agent executes it, the developer governs it.

Code Review

Automated review amplifies human judgment — and surfaces the graduation signals you've earned.



Trust Amplification

Security, logic, coverage — the answer to is the code any good?



review-complete

Final state machine transition: the loop closes here



Graduation Signals

Threshold indicators that the next SDLC phase is within reach

Time to review an agent-generated PR end-to-end
↳ 2-4 hrs → 20 min

Four Dimensions of Automated Review



Security Analysis

OWASP pattern detection, dependency vulnerability scanning, auth logic review



Logic Review

Business rule verification, edge case detection, race condition identification



Performance Checks

N+1 query detection, unnecessary re-renders, algorithmic complexity flags



Coverage Gates

Test coverage thresholds enforced — PRs fail CI if below plan targets



Automated review answers the first question — humans answer the architectural question.

From 4-Hour Reviews to 20-Minute Sign-Offs

Manual Review

2-4 hours per agent PR

Reviewers read 500-2000 lines of generated code

Security gaps caught post-merge — costly remediation

Inconsistent review depth across different reviewers

10-15x volume increase

Agent PRs arrive faster than humans can review

Automated Review

20 min human review

Automated pass surfaces all first-order issues before human reads a line

Security, logic, and coverage checked pre-merge

Consistent rubric — same depth on every PR

Reviewers shift roles

From line-by-line gatekeepers to architectural judges

83%

faster human review

0

post-merge security bugs

3.1x

PR throughput increase

Graduation Signals: You've Already Won — Now Scale



Cross-Repo Coordination

Agent changes spanning 2+ repos per feature exceed what this pipeline handles cleanly

Agents need shared context across boundaries

Monorepo or ACP manifest needed

See: Agentic SDLC talk



5+ Teams Participating

Coordination overhead starts exceeding automation gains at multi-team scale

Conflicting intake routing rules appear

Shared planning context required

Enterprise Patterns talk applies



Proven ROI on Record

You have 3+ months of time savings data — now is the moment to request the full transformation

Estimate accuracy tracked

Review throughput measured

Executive support earned

From Manual SDLC to Compounding Automation

Before

30 min triage per issue, 40 min per duplicate

2-4 hour ad-hoc planning with missed context

8-12 hour implementations with scope drift

2-4 hour code review with inconsistent coverage

After

3 min AI triage with duplicate detection and auto-labeling

15 min codebase-aware plan, human-gated via /approve-plan

1-2 hour agent-coded PR with tests and description

20 min automated security, logic, and coverage review

90%

less triage time per issue

94%

faster structured planning

4x

implementation throughput

✓ What You Can Do Today

Today

- Copy 4 YAML workflow files into your repo
- Configure secrets for GitHub Copilot agent access
- Create your first intake issue to test the pipeline

This Week

- Review and approve your first agent-generated execution plan
- Validate a Copilot-coded PR against your quality bar
- Measure triage time before and after for one sprint

This Month

- Track estimate accuracy improvements across 10+ planned issues
- Expand the workflow to a second repository
- Assess whether graduation signals are appearing in your team

🔑 Key Takeaway

Each phase saves time independently — but the ROI multiplies when all four run in sequence.

 Official Documentation[GitHub Copilot documentation](#)

Full reference for GitHub Copilot features and configuration

[Copilot coding agent for pull requests](#)

How the coding agent handles autonomous implementation

 Related Talks[Agent Teams](#)

Multi-agent orchestration patterns for complex engineering workflows

[Agentic SDLC](#)

Full software delivery lifecycle transformation with agentic workflows



The Agentic Journey

Incremental Path from Issue to PR Automation

3 min

AI triage replaces 30-minute manual intake
— duplicate detection included

/approve-plan

One human comment is the complete
answer to what if it goes wrong?

1-2 hrs

Agent-coded PR with tests appears —
zero implementation keystrokes

Which phase of the state machine would save your team the most time this sprint?